

NLQuery User Manual

Version 0.1.0a1 | September 2026 | Python 3.11+

1. What NLQuery does

NLQuery converts a question into an inspectable, typed query proposal. A language model interprets the request; deterministic software grounds references, plans operations, generates backend syntax, checks policy, and optionally executes. The model never serves as a database driver or security authority.

The normal path is question, bounded schema retrieval, typed intent, semantic grounding, logical plan, canonical IR, backend compiler, independent validation, explicit execution, and normalized result. A valid typed intent can enter the pipeline directly, without a language model or any model API access.

This is an alpha release. SQLite and PostgreSQL are the relational reference implementations. Other connectors have narrower capabilities and different levels of integration evidence. HANA execution is disabled. Read the backend maturity matrix and LIMITATIONS.md before choosing a deployment target. Unsupported requests fail rather than becoming plausible but incorrect backend operations.

2. Installation and first run

Create a virtual environment and install a release wheel or the tagged repository source. The core package depends only on Pydantic. SQLite is included in Python. Database drivers and LLM clients are optional extras, and constructing an adapter does not connect to a service.

```
python -m venv .venv
# Activate the environment using your operating system's normal command.
python -m pip install ./nlquery-0.1.0a1-py3-none-any.whl
python -m pip check
```

For source installation with PostgreSQL and a local model:

```
python -m pip install "nlquery[postgres,ollama] @
    git+https://github.com/drcrael/nlquery.git@v0.1.0a1"
```

Other extras are mysql, neo4j, mongo, redis, influx, hana, jira, openai, anthropic, yaml, and all. The all extra includes commercial drivers and may not be available on every platform. Use only the extras you need. This release is distributed through GitHub; no PyPI publication is implied.

Run examples/compile_only.py for a zero-network proposal. Run examples/ecommerce.py for the complete deterministic acceptance scenario, including a temporary database and exact assertions. The latter uses a caller-supplied mock intent, explicitly identified as a test fixture rather than model inference.

3. Your first proposal

The simplest offline workflow supplies a normalized schema and a typed intent. A `SourceSchema` represents an entity; `FieldSchema` records a property and type. `StaticConnector` permits planning and compilation but never execution.

```
from nlquery import NLQuery, QueryIntent
from nlquery.cli.main import offline
from nlquery.core.models import (
    FieldSchema,
    Filter,
    SchemaModel,
    SourceSchema,
)

schema = SchemaModel(
    backend="sqlite",
    sources=[
        SourceSchema(
            name="orders",
            fields=[
                FieldSchema(name="status", data_type="string"),
            ],
        ),
    ],
)

client = NLQuery(offline(schema))
compiled = client.compile(
    QueryIntent(
        sources=["orders"],
        filters=[Filter(field="status", value="paid")],
        limit=10,
    )
)

print(compiled.generated_query)
print(compiled.parameters)
```

The generated operation binds `paid` as a parameter. Values do not become SQL syntax. Identifiers come from validated schema metadata, and unsupported identifier forms are rejected. A proposal contains generated operation, parameters, logical plan, explanation, confidence components, provenance, risk warnings and referenced schema objects.

Inspect a proposal before execution. Confirm the selected source, fields, aggregation, grouping, time interval, ordering, and row limit. Deterministic validity cannot prove that a model selected the business interpretation you intended.

4. Connecting and executing

SQLiteConnector accepts an existing database path. It opens read-only connections and applies an authorizer plus a VM deadline. It will not create or populate your database. Application setup and migrations are outside this library's query interface.

PostgresConnector accepts a connection string separately from model input. Prefer `PostgresConnector.from_env("DATABASE_URL")` so credentials do not appear in scripts. PostgreSQL execution uses a read-only transaction and a statement timeout. Use a database account that is independently restricted to the permitted schema and operations.

```
from nlquery import NLQuery
from nlquery.connectors import PostgresConnector
from nlquery.llm.providers import OllamaProvider

client = NLQuery(
    PostgresConnector.from_env(),
    llm=OllamaProvider(model="qwen3"),
)
proposal = client.ask("Show the ten newest orders")
validation = client.validate(proposal)
if validation.allowed:
    result = client.execute(proposal)
    print(result.data)
```

`ask(question)` does not execute. `ask(question, execute=True)` explicitly requests both compilation and execution. `compile`, `validate` and `explain` never run the proposed operation, although schema discovery may read backend metadata. Supply a static schema for entirely offline compilation.

Execution rediscovers schema and independently reconstructs the operation under the current policy. An edited SQL string, changed parameter, newly prohibited source, or tighter limit invalidates the proposal. A serialized proposal is not an authorization token.

5. Typed intent and the logical plan

QueryIntent supports projections and aliases, predicates, boolean logic, metrics, dimensions, ordering, limits, offsets, declared joins, graph traversal, time windows, document unwind, distinct and restricted arithmetic. Each connector advertises which constructs it supports. A representable intent is not necessarily supported by every backend.

Filters use a field, operator and data value. Comparisons include equality/inequality, ordered comparisons, membership, null/existence tests, contains and relative_time. Membership lists must be nonempty, bounded and contain no nulls. Null and existence predicates take boolean values. Numeric aggregation requires known numeric schema metadata.

Predicate recursively combines filters using and, or or not. NOT has one child. Joins name declared schema relationships; arbitrary ON expressions are not accepted. Traversal names a graph relationship and has a finite minimum and maximum depth. Calculations allow only arithmetic between grounded numeric fields; arbitrary function strings are unsupported.

A QueryPlan holds the grounded QueryIR and an ordered list of logical operations such as Scan, Join, Traverse, Filter, Window, Aggregate, Project, Sort and Limit. Inspect and serialize with model_dump or model_dump_json. Reconstruct using QueryPlan.model_validate_json. Relative dates are already resolved in a compiled plan, so revalidation does not silently shift its time interval.

6. Business semantics, confidence and clarification

A semantic catalog makes business vocabulary explicit. A revenue concept can select `orders.total_amount`; an alias such as `sales` can map to the same concept. If multiple available candidates remain, the planner returns ambiguity instead of guessing. An explicit preferred candidate can resolve the choice.

```
from nlquery.core.models import SemanticCatalog, SemanticConcept

catalog = SemanticCatalog(
    concepts={
        "revenue": SemanticConcept(
            candidates=["orders.total_amount"],
            aliases=["sales"],
            units="USD",
        ),
    }
)
```

Pass the catalog to NLQuery. Catalog JSON and optional safe YAML files can be loaded through `nlquery.config.load_catalog`. Conditions are typed filters, never Python expressions. Descriptions, units, time semantics and source metadata are preserved but do not automatically execute currency conversions or arbitrary derived formulas.

Confidence is structured into deterministic evidence components: exact schema names, semantic mappings, type compatibility, declared relationships, calendar resolution and backend capability checks. It is not a calibrated probability of correct language understanding. A score of 1.0 means the recorded deterministic checks succeeded; it does not authorize execution or settle an unstated business definition.

`AmbiguityError` exposes candidates through its `ambiguities` attribute. Unknown critical fields and unresolved references block execution regardless of confidence. Clarify the question, add a catalog definition, or submit an explicit typed intent. Do not suppress validation to force an uncertain request through.

7. Time, document, graph and key/value semantics

Relative dates resolve against an injectable UTC clock at minute precision. Supported expressions include `last_quarter`, `previous_quarter`, `this_quarter`, `this_month`, `last_month`, `this_year`, `last_year`, `this_week`, `last_7_days`, `last_14_days` and `last_30_days`. Intervals are half-open: start inclusive, stop exclusive. Quarter/year rollover tests cover calendar boundaries. Business calendars and user timezones require an application-level policy.

Neo4j plans use node labels and declared relationship types. Traversal depth is finite and validated before dispatch. Native graph semantics are not translated into pretend relational joins. Multi-edge traversal chains and polymorphic edge inference require future support.

MongoDB plans preserve nested paths and arrays. Pipelines are structured objects containing supported `match/project/group/sort/limit/unwind` stages. No JavaScript is emitted. Date predicates are converted to native datetimes for fields discovered as dates. Sampling may miss rare fields; schema drift fails safely at execution. Lookup and arbitrary expressions remain unsupported.

InfluxDB explicitly supports version 2 Flux. A measurement requires bucket and time metadata, a start/stop range, and supported field/tag semantics. One metric and an optional aggregate window are supported. Automatic bucket discovery and versions 1/3 are not implemented.

Redis queries operate on explicit configured keys. Hash reads require selected fields; sorted sets use bounded ranges; streams use `COUNT`; set reads are random unordered samples. RedisJSON and RediSearch require installed modules. There are no uncontrolled global key scans or model-selected commands. Returned row limits do not guarantee small individual values.

Jira queries compile validated fields and predicates into JQL and retrieve issues through bounded enhanced-search pagination. JQL has no SQL-style parameter binding; string literals use a restricted encoder. Unsupported aggregation and literal-substring semantics are rejected. Qualify custom fields, dates and tenant-specific JQL rules against your own test tenant.

8. Policy and safe operations

QueryPolicy defaults to read-only, 1000 maximum rows, 30 seconds, graph depth 5, at most four joins, a bounded offset, and no cross-backend federation. `allowed_sources` optionally restricts entity access. Policies come from application configuration, not model output.

```
from nlquery import QueryPolicy

policy = QueryPolicy(
    max_rows=1000,
    max_execution_seconds=30,
    max_graph_depth=5,
    allowed_sources=["orders", "customers"],
)
```

Queries exceeding a bound are rejected rather than silently rewritten. Limits are checked again at execution. A row limit is not a scan-cost limit, memory limit or maximum response byte size. Native timeouts vary by backend; deploy server quotas and least-privileged accounts as independent safeguards.

Schema comments and sample metadata are untrusted data in a separate context object. The core never sends connector/provider credentials to the model. Known configured secrets and credential-shaped strings are rejected in artifacts and results. Unknown arbitrary secrets cannot be reliably detected; curate metadata and avoid exposing sensitive business data in prompts.

Raw backend exceptions are suppressed in public errors. NLQuery's logs contain stage, backend, elapsed time, request ID and risk, not parameter values. Third-party SDK debug logging is outside this guarantee. Compiled parameters are intentionally inspectable and may contain sensitive business values; protect stored proposals accordingly.

9. Multi-turn refinement and caching

QuerySession maintains previous intent and plan rather than an unlimited raw transcript. A refinement is interpreted as a complete replacement intent and receives the normal planning and validation checks. The session API does not automatically compare periods, federate systems or merge result sets.

Schema metadata is cached per client with a configurable TTL. `client.schema(refresh=True)` invalidates a discovered cache entry. Supplied static schema is copied rather than mutated. No query result caching occurs by default. There is no shared global mutable cache.

SchemaRetriever is a replaceable protocol. The built-in lexical retriever considers names, descriptions and catalog aliases, selects a limited number of sources, and enforces a context size budget. A deployment can implement vector retrieval behind the same contract without modifying deterministic compilation.

10. Command-line workflows

The CLI supports inspect, compile, ask, explain and validate. Use `--format json` for machine-readable output. Text mode currently also prints indented JSON. Read input intent and schema from files to avoid shell quoting mistakes.

```
nlquery inspect --database shop.db --format json
nlquery compile --database shop.db --intent intent.json --format json
nlquery explain --schema schema.json --intent intent.json
nlquery validate --database shop.db --compiled proposal.json
nlquery ask --database shop.db --llm ollama:qwen3 --execute "Show ten orders"
```

Only ask accepts `--execute`. Compile and explain refuse that flag. Offline `--schema` supports every registered built-in compiler; live CLI connection configuration currently handles SQLite and PostgreSQL. Other adapters are available through the Python API.

Configuration files contain policy/discovery settings, never credentials. `NLQUERY_MAX_ROWS` overrides a configured row bound. `NLQUERY_LLM` supplies the provider:model selector. `DATABASE_URL`, `OPENAI_API_KEY` and `ANTHROPIC_API_KEY` are read separately. Exit status 2 indicates a safe configuration, validation or execution failure without printing raw connection details.

11. Testing and extending

The normal suite requires no paid model API or commercial database. It includes compiler cases, exact SQLite results, opt-in PostgreSQL/MongoDB/Neo4j/Redis integration tests, transport contracts, security/property tests, 50 golden natural-language fixtures and cross-backend canonical-plan checks.

The golden corpus replays explicit mock intents. This validates deterministic behavior, not model interpretation accuracy. Use `scripts/evaluate.py` to evaluate a real provider against the version-controlled expectations. The optional live-model workflow records actual intents and SQLite replay outcomes for two paraphrases. Failures are retained as evidence; passing a narrow smoke test does not establish general accuracy.

To add a backend, implement the public Connector and QueryCompiler protocols, bounded discovery, safe credentials/error behavior, and independent execution revalidation. Advertise only tested capabilities. Use the extension walkthroughs in `docs/adding_connectors.md` and `docs/adding_compilers.md`; the first example builds a working offline connector without modifying core code.

12. Troubleshooting and release scope

Missing optional dependency: install the specific backend/provider extra. Unknown field: inspect discovered schema and check aliases. Ambiguous concept: supply a preferred catalog mapping or clarify the request. Unsupported capability: simplify the intent or select a backend that supports it. Proposal mismatch: recompile after schema or policy changes rather than editing generated query text. Query deadline: narrow the query and inspect server indexes/quotas; do not disable safety checks reflexively.

The alpha does not include HANA execution, InfluxDB 1/3, federation, full index/comment discovery, automatic business-calendar handling, arbitrary derived formulas, native EXPLAIN, async execution, vector retrieval, or general conversational analytics. These are explicit limitations, not hidden fallback paths.

Read `VALIDATION.md` for the exact tests and services actually exercised for this release. Read `SECURITY.md` before deploying against sensitive data. Read `docs/maturity.md` to distinguish compiler, contract and live-service evidence. Use the latest release and report reproducible issues with synthetic data and safe configuration details only.